CLIENT INTEGRATION SPECIFICATION

# MEDSYS Backend Client Specification

Client-facing contract guide for the MEDSYS API Enterprise backend. This document summarizes the implemented API surface, authentication flow, authorization model, request and response conventions, and the integration rules frontend and client teams should follow.

**DOCUMENT TYPE**
As-built client integration specification

**SYSTEM**
MEDSYS API Enterprise Backend

**VERSION**
3.2

**DATE**
April 6, 2026

**AUDIENCE**
All medical field professionals, technical managers, frontend, QA, BA, and client delivery teams

**SCOPE**
Implemented REST contracts, workflow behavior, permission model, integration expectations, and current handover-ready implementation state

This document describes the current implemented backend behavior and replaces older role-only guidance for UI authorization decisions.

## 1. Purpose

This specification gives client teams one stable reference for how to integrate with the MEDSYS backend today. It focuses on implemented behavior rather than future proposals, so frontend and integration teams can build against the live contract without inferring rules from role names or older screenshots.

## 2. Integration Snapshot

| AREA | CURRENT CONTRACT |
|---|---|
| Base API prefix | `/v1` |
| Primary content type | `application/json` |
| Access authentication | Bearer access token in the `Authorization` header. |
| Refresh flow | Refresh token is posted to `POST /v1/auth/refresh` to rotate and issue a new token pair. |
| Current-user endpoint | `GET /v1/auth/me` returns the authenticated user profile plus effective permissions. |
| Swagger UI | `/api/v1/docs` with request-body examples for implemented create and update flows. |
| Response style | JSON objects with snake_case for serialized fields such as `created_at`, `date_of_birth`, and `extra_permissions`. |

## 3. Identity and Authorization Model

> **Important:** the backend no longer expects clients to derive permissions only from the role name. Role and permissions are separate concepts. UI decisions should follow the returned permission list.

| CONCEPT | MEANING | CLIENT IMPACT |
|---|---|---|
| **role** | Legacy compatibility field aligned to the current `active_role`. | Keep supporting older clients, but treat `active_role` as the new source of truth. |
| **roles** | All assigned identity categories for the user, such as `["owner", "doctor"]`. | Use to understand workspace-switch options and multi-role behavior. |
| **active_role** | The current workspace context selected for the user session. | Use this field to decide which high-level workspace to render. |
| **permissions** | Effective allowed actions for the current user, resolved across assigned roles plus extra permissions. | Use this array to decide which screens, buttons, and flows should be available. |
| **workflow_profiles** | Operational behavior by role. Today this includes doctor mode plus standard owner/assistant defaults. | Use this object for workflow-aware UI behavior instead of inferring from permissions. |
| **doctor_workflow_mode** | Legacy compatibility field that mirrors `workflow_profiles.doctor.mode`. | Keep existing doctor clients working during migration. |
| **extra_permissions** | Per-user permission overrides stored on the user account. These are currently supported for selected doctor accounts only. | Useful for admin views and auditability, but UI authorization should still rely on the resolved `permissions` array. |

### 3.1 Effective Permission Resolution

The backend resolves a user's effective permissions centrally as:

`effective permissions = union of assigned role permissions + extra_permissions`

This means a multi-role user such as `["owner", "doctor"]` can switch workspace with `active_role` while still receiving the effective permission union expected by current backend authorization.

### 3.2 Assistant-Support Permission Subset

Owners can currently grant the following extra permissions to doctor accounts:

- `family.write`
- `inventory.write`

Doctors now receive `patient.write`, `appointment.create`, and `prescription.dispense` by default as part of the base doctor workflow. Owner-only permissions such as user administration or audit access are not part of this model and should not be expected on doctor accounts.

## 4. Authentication and Session Contracts

| ENDPOINT | PURPOSE | CONTRACT NOTES |
|---|---|---|
| `POST /v1/auth/login` | Authenticate and issue tokens | Returns `accessToken`, `refreshToken`, `expiresIn`, and a `user` object that includes `roles`, `active_role`, `workflow_profiles`, legacy `role`, legacy `doctor_workflow_mode`, `permissions`, and `extra_permissions`. |
| `POST /v1/auth/refresh` | Rotate refresh token | Use when the access token expires. The refreshed response also returns the current permission set. |
| `GET /v1/auth/me` | Load current session profile | Frontend should call this after app bootstrap or session restore and treat it as the source of truth for permissions. |
| `POST /v1/auth/active-role` | Switch workspace role | Authenticated users can switch to any assigned role. Backend persists the choice in `active_role`. |
| `POST /v1/auth/logout` | Revoke active refresh tokens | Authenticated route. |
| `GET /v1/auth/status` | Bootstrap status | Unauthenticated helper for initial setup state. |
| `POST /v1/auth/register` | Create a user or bootstrap first owner | Supports standard user creation and doctor accounts with optional extra permissions beyond the default doctor workflow. |

### 4.1 Login Response Shape

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresIn": 900,
  "user": {
    "id": 12,
    "user_id": 12,
    "name": "Support Doctor",
    "email": "doctor-support@example.com",
    "role": "doctor",
    "roles": ["owner", "doctor"],
    "active_role": "doctor",
    "doctor_workflow_mode": "clinic_supported",
    "workflow_profiles": {
      "doctor": { "mode": "clinic_supported" },
      "assistant": null,
      "owner": { "mode": "standard" }
    },
    "permissions": [
      "appointment.read",
      "appointment.create",
      "appointment.update",
      "encounter.read",
      "encounter.write",
      "inventory.read",
      "patient.read",
      "patient.write",
      "prescription.read",
      "prescription.dispense"
    ],
    "extra_permissions": [
      "inventory.write"
    ]
  }
}
```

### 4.2 Frontend Authorization Rule

Frontend clients should render actions based on the returned `permissions` array, render workspaces from `active_role`, and use `workflow_profiles` for role-specific workflow behavior. Legacy fields remain available for compatibility only.

## 5. User Management Contract

| ENDPOINT | PURPOSE | CONTRACT NOTES |
|---|---|---|
| `GET /v1/users` | List users | Returns each user with `roles`, `active_role`, `workflow_profiles`, legacy `role`, legacy `doctor_workflow_mode`, `permissions`, and `extra_permissions`. |
| `POST /v1/users` | Create user | Supports both frontend-style `name` payloads and direct API `firstName/lastName` payloads. Owners can create users with one or more assigned roles, optional `activeRole`, and doctor workflow identity when the doctor role is present. |
| `PATCH /v1/users/:id` | Update activation, roles, active role, doctor workflow mode, or extra permissions | Used by owners to manage role assignments, `activeRole`, allowed doctor workflow identity, and the remaining extra-permission subset. |

### 5.1 Example Permission Assignment

```
{
  "doctorWorkflowMode": "clinic_supported",
  "extraPermissions": [
    "inventory.write",
    "family.write"
  ]
}
```

Only doctor users can receive `doctorWorkflowMode`, `activeRole` must be one of the assigned `roles`, and only valid permission names are accepted. Unsupported permission names or owner-only permissions are rejected by the backend.

## 6. Domain Endpoint Summary

| DOMAIN | PRIMARY ROUTES | KEY PERMISSIONS | NOTES |
|---|---|---|---|
| Patients | `/v1/patients` | `patient.read`, `patient.write`, related history, allergy, condition, vital, and timeline permissions | `GET /v1/patients` is a summary-list contract. `GET /v1/patients/:id/profile` is the full overview bundle, and `GET /v1/patients/:id/consultations` is the consultation-history contract for encounter-first UI cards. |
| Families | `/v1/families` | `family.read`, `family.write` | Doctor accounts can receive `family.write` through assistant-support permissions. |
| Appointments | `/v1/appointments` | `appointment.read`, `appointment.create`, `appointment.update` | Appointment creation is no longer tied only to the assistant role. It follows permission checks and remains the resource-level route for scheduled visits. |
| Visits | `/v1/visits/start` | `appointment.create` | Walk-in orchestration route that reuses an active patient visit or creates a new in-consultation visit for doctor-first workflows. |
| Encounters | `/v1/encounters` | `encounter.read`, `encounter.write` | Doctor base permissions already cover clinical encounter authoring. |
| Prescriptions | `/v1/prescriptions` | `prescription.read`, `prescription.dispense` | Doctors can access the pending dispense queue and complete dispense workflow under the default doctor permission set. |
| Inventory | `/v1/inventory` | `inventory.read`, `inventory.write` | Mutation routes are permission-checked; doctors can create inventory items, item detail updates stay separate from stock movements, stock-adjust and batch routes are available, and alerts plus reports support operational stock control. |
| Clinical lookup | `/v1/clinical` | `clinical.icd10.read` | Reference read flows for normalized diagnosis search, test search, curated recommended tests, and backward-compatible ICD-10 search. |
| Analytics | `/v1/analytics` | `analytics.read` | Operational dashboards and cache metrics. Overview keeps additive `role_context`, while `GET /v1/analytics/dashboard` is the richer role-aware dashboard contract. |
| Audit | `/v1/audit` | `audit.read` | Reserved for users with audit visibility, typically owners. |
| Search | `/v1/search` | `patient.read` | Patient search surface with accelerated and fallback behavior. |

## 7. Validation and Data Rules Important to Clients

- Patient `first_name` and `last_name` are required. Therefore the stored generated `full_name` is also never null.
- Patient `dob` is required and `age` is also required. The backend derives and validates age against DOB when provided.

- Patient `nic` is optional. The backend now assigns every patient a mandatory `patient_code` used as the always-available internal identifier.
- Patient API responses use snake_case fields such as `patient_code`, `date_of_birth`, `family_id`, `guardian_patient_id`, `allergy_highlights`, and `created_at`. Clients should not rely on temporary camelCase response fields.
- Minor patients without a patient NIC must include guardian information. Clients can either link an existing guardian with `guardianPatientId` in the write payload or provide guardian name plus guardian NIC or phone.
- Guardian-aware patient write payload fields supported by create and update flows are `familyId`, `familyCode`, `guardianPatientId`, `guardianName`, `guardianNic`, `guardianPhone`, and `guardianRelationship`.
- Patient create supports `bloodGroup` and inline `allergies` in both request styles. Allergy rows use `allergyName`, `severity`, and `isActive`, where severity must be `low`, `moderate`, or `high`.
- If neither `familyId` nor `familyCode` is provided on patient create, the backend auto-creates a family named from the patient, such as `Nimal Perera Family`.
- `priority` remains an appointment-only field. Clients should send visit urgency to `POST /v1/appointments`, not to the patient master record.
- Vitals writes use a fixed typed payload, not a generic name or value array. Supported fields are `bpSystolic`, `bpDiastolic`, `heartRate`, `temperatureC`, `spo2`, optional `encounterId`, and required `recordedAt`.
- `POST /v1/patients/:id/vitals` requires at least one actual measurement value. `recordedAt` by itself is not enough.
- Vitals responses are serialized in stable snake_case fields such as `patient_id`, `encounter_id`, `bp_systolic`, `heart_rate`, `temperature_c`, `recorded_at`, `created_at`, and `updated_at`.
- `PATCH /v1/patients/:id/vitals/:vitalId` corrects an existing vitals row, and `DELETE /v1/patients/:id/vitals/:vitalId` soft-deletes a mistaken row.
- If `encounterId` is sent with a vitals create or update request, the backend validates that the encounter belongs to the same patient and organization.
- `POST /v1/encounters` now supports an optional `vitals` block so the consultation workflow can create the encounter and initial vitals together. If the vitals block omits `recordedAt`, the backend uses `checkedAt`.
- `PATCH /v1/patients/:id` is transactional for patient profile, family, and allergy changes. If any part of a combined update fails, the full save is rolled back.
- `POST /v1/visits/start` is the preferred walk-in flow endpoint. It reuses an active visit in `waiting` or `in_consultation` for the patient, or creates a new visit if none exists.
- User creation supports two request shapes: frontend-friendly `name` or direct API `firstName/lastName`.
- Swagger examples are available on the major create and update endpoints to reduce ambiguity around payload shapes.
- Permission names are validated. Clients should not send arbitrary strings in `extraPermissions`.
- Inventory uses a base-unit stock model. `unit` is the base stock unit, while `dispenseUnit` / `dispenseUnitSize` and `purchaseUnit` / `purchaseUnitSize` describe packaging conversions for clinic dispense and supplier purchase workflows.
- `PATCH /v1/inventory/:id` updates inventory item details, packaging rules, thresholds, and flags. It does not change the live stock balance.
- Inventory quantity changes should use `POST /v1/inventory/:id/movements`. Frontend can now send `movementUnit` using the configured base, dispense, or purchase unit and let backend convert to base stock automatically.
- `POST /v1/inventory/:id/adjust-stock` is the correct endpoint for “actual counted stock is X” corrections.
- `GET /v1/inventory/:id` returns item detail, stock summary, and recent movement context, while `GET /v1/inventory/:id/batches` and `POST /v1/inventory/:id/batches` support lot-level expiry and supplier tracking.
- `GET /v1/inventory/alerts` returns low-stock, stockout-risk, and expiry-aware alert rows plus reorder recommendations based on recent outgoing movement history, and `GET /v1/inventory/reports` returns supplier summary, movement velocity, and expiring batch data.

### 7.1 Inventory Actions In Simple Terms

- **Edit Item Details**: change the item name, supplier, packaging setup, minimum stock level, and notes.
- **Add Stock**: record newly received stock.
- **Remove Stock**: record stock that was used, dispensed, damaged, or expired.
- **Correct Stock Count**: set the system to the real counted quantity when physical stock and system stock do not match.
- **Add Batch**: record a new lot or batch with its own expiry date and storage location.

### 7.2 Operational Language Guide

- `movementUnit` should be shown to staff as **Stock Unit Used**.
- `adjust-stock` should be shown to staff as **Correct Stock Count**.
- `stockSummary.currentStock` should be shown as **Current Stock**.
- `stockSummary.minimumStock` should be shown as **Minimum Stock Level**.
- `stockSummary.shortageToMinimum` should be shown as **Short by**.

- `stockSummary.purchasePackEquivalent` should be shown as **Equivalent Purchase Packs**.
- `stockSummary.dispensePackEquivalent` should be shown as **Equivalent Dispense Packs**.
- `inventory reports` should be described as **Inventory Summary and Warnings**.

### 7.3 Simple Staff Examples

- **Current Stock:** 200 tablets
- **Minimum Stock Level:** 50 tablets
- **Short by:** 10 tablets
- If 2 boxes are received and 1 box = 100 tablets, the system adds 200 tablets automatically.
- If physical counted stock is 180, staff should use **Correct Stock Count** instead of manually calculating the difference.

## 8. Recommended Frontend Behavior

1. Load `GET /v1/auth/me` on session restore and keep the returned `permissions` list as the UI authorization source.
2. Use `active_role` for workspace selection, `roles` for switcher options, and `role` only as a legacy compatibility alias.
3. Hide or disable create, update, dispense, and inventory mutation actions when the required permission is absent.
4. Expect a multi-role user to keep the full assigned `roles` set while `role` mirrors the currently selected `active_role`.
5. Use `GET /v1/patients` for summary lists, `GET /v1/search/patients` for doctor fast-selection by patient/guardian identity (returns `patient_code`, `nic`, `gender`), `GET /v1/patients/:id/profile` for the full patient overview bundle, and `GET /v1/patients/:id/consultations` for encounter-grouped consultation history cards.
6. `GET /v1/patients` now returns a richer card summary including `family_name`, `visit_count`, `last_visit_at`, `next_appointment`, `allergy_highlights`, and `major_active_condition` so patient list screens do not need per-row fan-out requests.
7. `GET /v1/patients/:id/profile` now includes a bundled `family` section with family header data and members, alongside `allergies`, `conditions`, `vitals`, and `timeline`.
8. Do not assume patient NIC is always present in list responses or even on the patient record. For minors, use guardian-aware search and treat `patient_code` as the always-available identifier.
9. For the doctor screen, prefer `POST /v1/consultations/save` as the one-button workflow endpoint. Use `workflowType = walk_in` for direct doctor flow and `workflowType = appointment` when saving against an existing appointment. Send `patientId` for an existing patient or `patientDraft` for inline quick-create in walk-in mode.
10. Use `patientDraft.name` plus `patientDraft.dateOfBirth` as the minimum quick-create fields. Treat date of birth as canonical and use age only as an optional helper.
11. For minors without a patient NIC, send guardian details in the same consultation request. If `guardianNic` matches an existing patient, backend will auto-link the guardian and inherit family linkage when possible. If frontend has enough guardian demographics to create a real guardian patient, send `guardianDraft`.
12. Use `clinicalSummary` when the doctor wants a narrative clinical note in patient history. Use `persistAsCondition = true` on a diagnosis row only when it should become part of the patient's long-term condition list.
13. Use `GET /v1/clinical/diagnoses` for diagnosis autocomplete, `GET /v1/clinical/tests` for test autocomplete, and `GET /v1/clinical/diagnoses/:code/recommended-tests` for backend-owned suggested tests. Frontend should not call external terminology vendors directly.
14. `GET /v1/clinical/tests` is backed by configured terminology providers and currently defaults to the public NLM Clinical Tables LOINC search endpoint. It is intended for lab tests and clinical observations, not as the final catalog for imaging procedures such as X-ray, CT, MRI, or ultrasound.
15. Prescription items now behave differently by `source`: `clinical` items drive internal dispense workflow, while `outside` items stay on the prescription and history only. Use `workflow_type`, `appointment_id`, `workflow_status`, `dispense_status`, `clinical_item_count`, and `outside_item_count` from consultation save responses to decide whether assistant handoff is required.
16. `GET /v1/appointments?status=waiting` now returns queue rows in backend-controlled FIFO order and includes an explicit `queuePosition` field. Frontend should use that value for queue numbering instead of deriving numbers from array index.
17. Use `GET /v1/prescriptions/queue/pending-dispense` for assistant handoff rows and `GET /v1/inventory/search?q=...&category=medicine` when a queued clinical item still needs stock matching before final dispense.
18. For inventory screens, show `Current Stock` using base units, keep `Minimum Stock Level` separate from `Recommended Reorder Qty`, render packaging labels only when configured, and prefer backend `stockSummary` values over frontend recalculation.
19. For small walk-in clinics using older resource-level orchestration, `POST /v1/visits/start` remains valid. For larger centers with receptionist-led scheduling, keep using `POST /v1/appointments` and the waiting queue.
20. Use Swagger examples as the canonical request-body reference when onboarding client teams.

## 9. Dual Clinic Workflow Support

The backend now supports both Sri Lankan walk-in clinics and appointment-first multi-doctor centers without changing the core patient, visit, encounter, and dispense model.

### 9.1 Walk-In Doctor Flow

1. Doctor searches patient using `GET /v1/search/patients` by code, NIC, phone, or name.
2. If the patient exists, frontend sends `POST /v1/consultations/save` with `patientId`.
3. If the patient does not exist, frontend stays on the same screen and sends `POST /v1/consultations/save` with `patientDraft`.
4. The same consultation request can carry diagnoses, tests, notes, vitals, prescription items, allergies, and an optional `clinicalSummary`. Walk-in mode can also send `dispense.mode = doctor_direct` to record immediate dispense; otherwise prescriptions remain pending for the dispense queue.
5. The backend creates or resolves the patient, reuses an existing active visit in `waiting` or `in_consultation` if present, otherwise creates a new walk-in visit, then saves the encounter atomically.
6. If an active walk-in consultation with an encounter already exists for the patient, backend now returns a clear `409 Conflict` response instead of failing with a generic server error.
7. The backend also creates a patient timeline event automatically, optionally stores `clinicalSummary` as a narrative history entry, and promotes any diagnosis marked with `persistAsCondition = true` into long-term patient conditions.
8. For minor patients without a patient NIC, guardian details are part of the same workflow, guardian NIC can auto-link an existing guardian patient record, and `guardianDraft` can be used when frontend wants backend to create the guardian patient and family linkage immediately.
9. If dispensing inside the clinic, doctor optionally completes the dispense workflow directly after consultation save.

### 9.2 Appointment-First Center Flow

1. Assistant or scheduling staff creates the appointment using `POST /v1/appointments` with the required visit metadata such as `scheduledAt` and optional `priority`.
2. Doctor works from the waiting queue and selects the queued patient. Queue numbering should follow the backend-provided `queuePosition` value.
3. Doctor can use `POST /v1/consultations/save` as the one-button clinical save flow. Appointment mode returns `workflow_type = appointment` plus `appointment_id`, while walk-in mode returns `workflow_type = walk_in` and `appointment_id = null`.
4. The same patient, visit, encounter, and dispense APIs remain available across both operating modes.

## 9. Source Basis

- Repository implementation state as of April 6, 2026.
- Auth and user management contract including effective permission responses and user-level permission overrides.
- Patient summary/profile separation, patient code identifiers, guardian-aware registration, walk-in visit orchestration, consultation workflow orchestration, fixed typed vitals workflows, doctor workflow identity, persisted multi-role support through Flyway V12, and current database constraints.
- Verified automated backend suite status of 92 passing API tests and 6 passing worker tests.

---

This document is the maintained client-facing reference for the implemented MEDSYS backend contract as of April 6, 2026.